

Workshop 3

Design Patterns Documentation

Students:

Juan David Castaneda Cardenas

Laura Vanesa Alvarez Lafont

Nicolas Andres Bolanos Fernandez

Santiago Andres Fetecua Pulgarin

Sergio Nicolas Correa Escobar

Professor:

Carlos Andres Sierra Virguez

**Software Engineering II
School of Engineering
Universidad Nacional de Colombia**

Strategy Pattern

The authentication microservice needs to validate different types of user credentials (username or email). Each authentication type has its own rules and criteria, so maintaining these variations directly in the service would increase code complexity. For this reason, we use the Strategy pattern, which allows changing or extending the authentication logic without modifying the main login flow.

The Strategy pattern encapsulates each authentication method in an independent strategy that implements a common interface (**CredentialStrategy**). The login service acts as the context, selecting the appropriate strategy based on the contents of the **LoginRequest**. This approach allows the system to switch between **EmailCredentialStrategy** and **UsernameCredentialStrategy**.

The interface **CredentialStrategy** defines the contract for all supported authentication methods. This interface includes two key operations:

- **supports(LoginRequest req):** Determines whether the strategy is applicable for the given request. This allows each strategy to indicate if it can handle the specific type of credential provided.
- **authenticate(LoginRequest req):** Performs the authentication process and returns an **Optional<User>** to indicate success or failure. Each strategy implements its own lookup and verification logic while adhering to the same method signature.

New authentication methods can be added simply by implementing the interface.

Two concrete implementations of the **CredentialStrategy** interface were created to support different types of user credentials:

- **EmailCredentialStrategy**
This strategy is selected when the identifier in the **LoginRequest** contains an "@", indicating that the user is attempting to authenticate with an email address.
- **UsernameCredentialStrategy**
This strategy is applied when the identifier does not contain an "@", meaning the user is logging in with a username.

Both strategies retrieve the corresponding user using either **findByEmail** or **findByUsername**, and validate the provided password using **BCryptPasswordEncoder**. On a successful match, the strategy returns the authenticated user.

The implementation codes are shown below:

Java

```
public interface CredentialStrategy {  
    boolean supports(LoginRequest req);  
    Optional<User> authenticate(LoginRequest req);  
}
```

Java

@Component

```
public class EmailCredentialStrategy implements CredentialStrategy {  
  
    private final UserRepository userRepository;  
    private final BCryptPasswordEncoder passwordEncoder = new  
BCryptPasswordEncoder();  
  
    public EmailCredentialStrategy(UserRepository userRepository) {  
        this.userRepository = userRepository;  
    }  
  
    @Override  
    public boolean supports(LoginRequest req) {  
        String id = req.identifier();  
        return id != null && id.contains("@");  
    }  
  
    @Override  
    public Optional<User> authenticate(LoginRequest req) {  
        return userRepository  
            .findByEmail(req.identifier())  
            .filter(u -> passwordEncoder.matches(req.password(),  
u.getPasswordHash()));  
    }  
}
```

Java

@Component

```
public class UsernameCredentialStrategy implements CredentialStrategy {
```

```

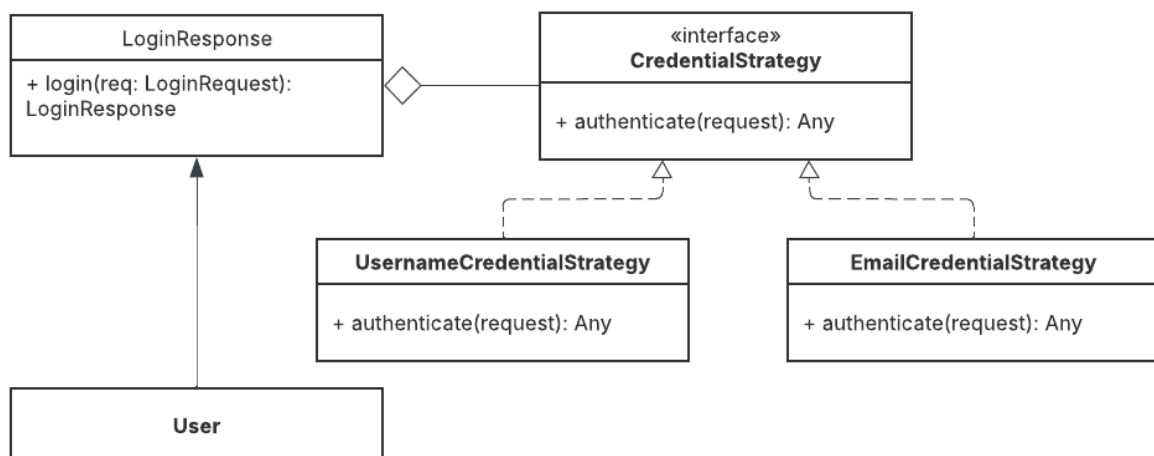
private final UserRepository userRepository;
private final BCryptPasswordEncoder passwordEncoder = new
BCryptPasswordEncoder();

public UsernameCredentialStrategy(UserRepository userRepository) {
    this.userRepository = userRepository;
}

@Override
public boolean supports(LoginRequest req) {
    String id = req.identifier();
    return id != null && !id.contains("@");
}

@Override
public Optional<User> authenticate(LoginRequest req) {
    return userRepository
        .findByUsername(req.identifier())
        .filter(u -> passwordEncoder.matches(req.password(),
u.getPasswordHash()));
}
}

```



Singleton Pattern

There are certain components in the system that we need to access constantly. Re-creating or re-loading them repeatedly would be a waste of resources, which is why we apply the Singleton pattern in the project.

This approach allows us to keep a single, shared instance that can be reused across the service whenever it is needed.

Although the classical Singleton pattern is not implemented explicitly (there is no private constructor or `getInstance()` method), the **event_manager** microservice effectively uses a Singleton-like mechanism through [SQLAlchemy](#).

In this project, the [SQLAlchemy](#) configuration is defined at the module level, and because of Python's module loading system, that module is initialized only once. As a result, every component that imports it shares the same database engine and session factory, which behaves similarly to a Singleton.

Specifically:

- **engine**: single database engine instance shared by all components
- **SessionLocal**: single session factory shared by all services
- **Base**: single model metadata registry used by all ORM models

This design ensures consistent database access across the microservice while preventing multiple engine creations, improving performance and maintaining a clean and centralized configuration.

The implementation code is shown below:

```
Python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "sqlite:///./events.db"

engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
```

```
        autoflush=False,  
        bind=engine  
    )  
  
    Base = declarative_base()
```